

PROJECT REPORT

ShaktiSOC

AI-Powered Security Operations Center using ShaktiDB

A full-stack cybersecurity telemetry and threat-detection platform,
built on ShaktiDB and delivered as a completed mini-project

Repository: [ras-al/shaktidb-work](#)

Prepared by:

Rasal Musthafa, 240236

B.Tech, Computer Science & Engineering

July 2026

TABLE OF CONTENTS

1. Introduction.....	3
2. Problem Statement.....	4
3. Objectives	5
4. System Architecture.....	6
5. Database Design.....	8
6. Technology Stack.....	9
7. Repository Structure	10
8. Implementation Details.....	11
8.1 Telemetry Collection Layer	11
8.2 AI Threat Detection Engine	11
8.3 Backend REST API	12
8.4 Web Dashboard.....	13
9. System Validation and Attack Simulation.....	14
10. Results and Screenshots.....	16
10.1 Database Schema	16
10.2 Dashboard in Operation	16
11. Development Timeline.....	18
12. Challenges and Solutions.....	19
13. Future Scope	20
Conclusion	21
Author Details	22

1. Introduction

Modern computing environments are exposed to a wide range of security threats that operate silently at the operating-system level - unauthorised logins, abnormal process behaviour, unusual network activity, and unexpected file changes. Detecting these events in real time and correlating them into a single, coherent picture of system health is the core function of a Security Operations Center (SOC). ShaktiSOC is a self-contained, full-stack implementation of this idea, purpose-built to run on a single Linux endpoint while demonstrating the same principles used in enterprise-grade security monitoring platforms.

The project was undertaken to explore ShaktiDB, an indigenous PostgreSQL-compatible database engine, as the storage and query backbone for a continuously running telemetry pipeline. Rather than treating the database as a passive component, ShaktiSOC was designed so that ShaktiDB sits at the centre of the system: every login attempt, process spike, network connection, and file-system change is written to it, queried from it, and eventually reasoned about by a machine learning model built on top of it. This report documents the finished system - its design, implementation, validation, and outcomes - based on the project repository, its architecture and proposal documents, the installation notes, and seven weekly development reports.

2. Problem Statement

Conventional security monitoring tools frequently rely on static, rule-based detection logic. Fixed thresholds and predefined signatures are effective against known attack patterns but tend to miss novel or evolving techniques, and they often generate large volumes of low-value alerts that overwhelm an operator - a problem commonly referred to as alert fatigue. There was, therefore, a need for a lightweight platform that could:

- continuously collect telemetry from several independent attack surfaces on a Linux host;
- store that telemetry efficiently in a relational database suited to time-series security data;
- analyse the resulting data using an unsupervised learning approach that does not depend on pre-labelled attack samples; and
- present the findings through a live, human-readable dashboard rather than raw log files.

ShaktiSOC was scoped to address each of these requirements directly, while simultaneously acting as a practical evaluation of ShaktiDB under a realistic, continuously-writing workload.

3. Objectives

1. Install, configure, and evaluate ShaktiDB as the primary data store for a security telemetry pipeline.
2. Design and build independent collector daemons for login, process, network, and file-system telemetry on Ubuntu Linux.
3. Implement a context-aware anomaly detection engine capable of distinguishing genuine threats from routine system noise.
4. Expose collected data and generated alerts through a documented REST API.
5. Build a real-time, browser-based dashboard that visualises system status, telemetry, and alerts.
6. Validate the complete pipeline end-to-end using a simulated multi-vector attack.
7. Maintain the codebase, documentation, and weekly progress records on GitHub throughout development.

4. System Architecture

ShaktiSOC follows a layered pipeline architecture in which data flows in one direction, from the operating system outward to the operator's browser. Four independent collector processes observe the host system and write structured events into ShaktiDB. A separate analytics process periodically reads this data, applies a machine learning model, and writes any resulting alerts back into the same database. A Flask-based REST API exposes both the raw telemetry and the generated alerts as JSON, and a React front end polls this API every five seconds to keep the dashboard current. The overall flow is summarised below.

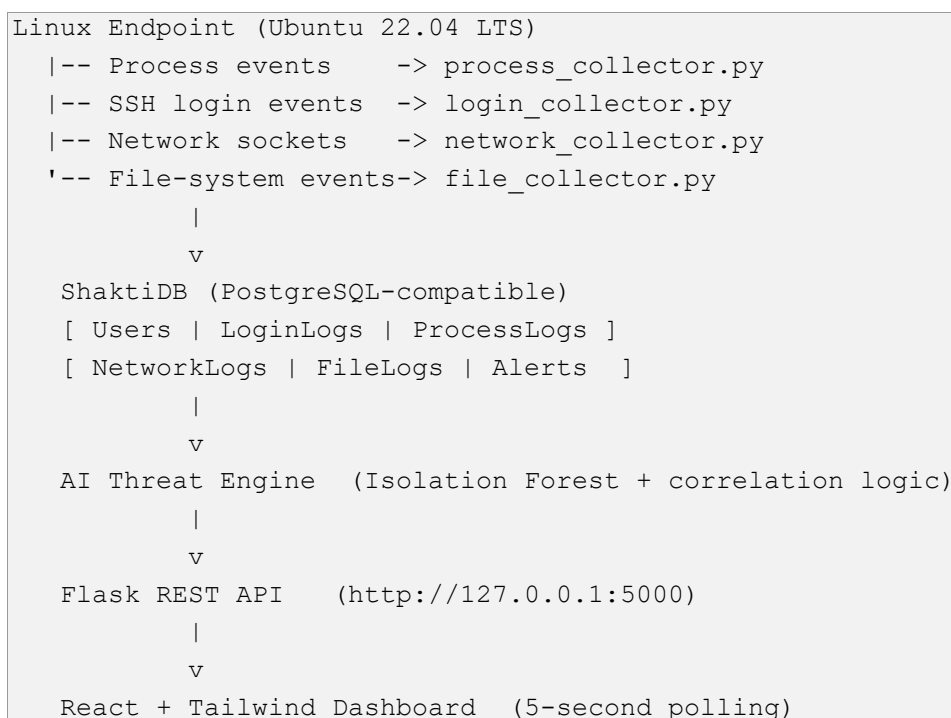


Figure 1: End-to-end data flow through the ShaktiSOC pipeline

4.1 Component Overview

- Telemetry Collection Layer - four Python daemons, each responsible for one attack surface (process activity, authentication, network sockets, and file integrity), running independently and continuously.
- Database Layer - ShaktiDB, storing all telemetry and alerts in six relational tables described in Section 5.

- Analytics Layer - a scheduled process that transforms stored telemetry into feature data using Pandas, scores it with a tuned Isolation Forest model, and correlates anomalies across vectors.
- API Layer - a Flask application exposing read endpoints for each log type and for alerts, with CORS enabled for the front end.
- Presentation Layer - a React and Tailwind CSS dashboard that renders live charts, log tables, and colour-coded alert panels.

5. Database Design

ShaktiDB was chosen as the sole data store for the project. Its schema was designed specifically for time-series security events, with each collector writing to its own dedicated table and the analytics layer writing derived alerts to a shared table. The six tables that make up the schema are summarised in Table 1.

Table	Purpose	Populated By
Users	Stores user accounts and role information.	Manual / setup
LoginLogs	Records SSH authentication attempts, including username, source IP, and outcome.	login_collector.py
ProcessLogs	Captures processes exceeding CPU or memory thresholds, with PID and resource usage.	process_collector.py
NetworkLogs	Logs active TCP/UDP sockets and their connection state.	network_collector.py
FileLogs	Records file creation, modification, and deletion events on watched paths.	file_collector.py
Alerts	Stores AI-generated security alerts with event type, description, and severity.	threat_detector.py

Table 1: ShaktiDB schema used by ShaktiSOC

Each table uses an auto-incrementing primary key and a timestamp column defaulting to the current time, which allows the analytics layer to query only records created since its last run rather than reprocessing the entire history on every cycle. This time-windowed query pattern was introduced specifically to prevent duplicate alert generation, a problem encountered and resolved during development.

6. Technology Stack

The finished platform draws on the following technologies, organised by functional layer.

Layer	Technology
Database	ShaktiDB (PostgreSQL-compatible)
Backend / Collectors	Python 3, psutil, watchdog
Machine Learning	Scikit-Learn (Isolation Forest), Pandas, NumPy
API Server	Flask 3.0, Flask-CORS
Frontend	React 19, Vite, Tailwind CSS 3
Charting and Icons	Recharts, Lucide React
Data Fetching	Axios (5-second polling)
Operating System	Ubuntu 22.04 LTS
Version Control	Git, GitHub

Table 2: Technology stack used across the ShaktiSOC platform

7. Repository Structure

The project source and supporting material are maintained in the GitHub repository [ras-al/shaktidb-work](https://github.com/ras-al/shaktidb-work). The repository separates application code from documentation, installation artefacts, and progress records, as summarised in Table 3.

Directory	Contents
shaktisoc/	Application source code: collectors, analytics engine, Flask API, database schema, and the React dashboard (shaktisoc-ui).
docs/	Architecture document, original project proposal, and system information notes.
reports/	Seven weekly progress reports covering the full development timeline.
installation/	ShaktiDB installer package, public key, and a step-by-step installation guide.
scripts/	An interactive shell utility for viewing system, CPU, memory, disk, and process information.
bugs/	Space reserved for documenting issues and observations encountered while working with ShaktiDB.

Table 3: Repository directory layout

8. Implementation Details

8.1 Telemetry Collection Layer

Four independent Python daemons form the sensing layer of ShaktiSOC, each targeting a distinct attack surface:

- `process_collector.py` polls running processes using `psutil` and writes an entry to `ProcessLogs` only when a process exceeds defined CPU or memory thresholds, which keeps the table from growing unnecessarily large.
- `login_collector.py` tails the system authentication log (`/var/log/auth.log`) and uses regular expressions to extract the username, source address, and outcome of every SSH login attempt, writing the result to `LoginLogs`.
- `network_collector.py` scans active TCP and UDP sockets with `psutil` and filters on connection state, so that `ESTABLISHED` and `LISTEN` sockets relevant to intrusion detection are captured in `NetworkLogs`.
- `file_collector.py` uses the `watchdog` library to watch sensitive directories such as `/tmp` asynchronously, logging every `CREATED`, `MODIFIED`, or `DELETED` event to `FileLogs` in real time.

Together, these four collectors give ShaktiSOC continuous visibility across authentication, process, network, and file-system activity without relying on a single monolithic monitoring agent.

8.2 AI Threat Detection Engine

The analytics layer, implemented as `threat_detector.py`, converts raw telemetry into a feature set using `Pandas` and applies an Isolation Forest model from `Scikit-Learn` to flag statistically unusual behaviour. Because the model is unsupervised, it does not require a labelled dataset of known attacks; instead, it learns the host's own baseline resource usage and flags deviations from it. Several refinements were added during development to make the engine practical rather than merely functional:

- Process whitelisting - common, resource-intensive but benign applications such as Chrome, Firefox, VS Code, and core system processes are excluded from anomaly scoring.

- Hard thresholds - a flagged anomaly must also exceed 10% CPU or memory usage before it is treated as a candidate threat, filtering out statistically unusual but practically harmless spikes.
- Low contamination rate - the model's contamination parameter is set to 1%, so only the most significant outliers are surfaced.
- Cross-vector correlation - if a CPU anomaly coincides with new file-system activity and new network connections within the same time window, the alert is escalated to CRITICAL severity as a likely coordinated attack rather than an isolated event.
- Time-windowed processing - a `last_check_time` filter ensures the engine only evaluates telemetry generated since its previous run, preventing duplicate alerts on historical data.

This combination of unsupervised learning with rule-based refinement was adopted specifically to reduce alert fatigue, a limitation identified in conventional rule-only monitoring tools during the problem statement.

8.3 Backend REST API

A Flask application (`api.py`) exposes the contents of ShaktiDB to the front end as JSON over a small set of read-only endpoints, listed in Table 4. Cross-Origin Resource Sharing is enabled so that the React dashboard, served from a different port during development, can query the API directly.

Method	Endpoint	Description
GET	<code>/api/status</code>	API health check.
GET	<code>/api/logs/processes</code>	Latest 50 process telemetry events.
GET	<code>/api/logs/logins</code>	Latest 50 SSH authentication events.
GET	<code>/api/logs/network</code>	Latest 50 network socket events.
GET	<code>/api/logs/files</code>	Latest 50 file system events.
GET	<code>/api/logs/alerts</code>	Latest 50 AI-generated alerts.

Table 4: REST API endpoints exposed by the Flask backend (<http://127.0.0.1:5000>)

8.4 Web Dashboard

The presentation layer is a single-page application built with React and Vite, styled with Tailwind CSS to give it a dark, enterprise-style appearance. Axios polls the backend every five seconds, and Recharts renders the live telemetry as an interactive line chart. The dashboard is organised into six panels, each corresponding to one of the database tables: a system status indicator, an AI Threat Engine alerts panel, a live CPU telemetry chart, an authentication log, a network socket state list, and a file integrity monitor. Lucide React supplies the iconography used throughout the interface.

9. System Validation and Attack Simulation

Rather than testing each module in isolation, the completed system was validated with a single end-to-end exercise designed to exercise every collector and the correlation logic at once. The exercise simulated a simplified multi-stage intrusion consisting of a failed SSH login, the creation of a suspicious file, an outbound network connection, and a sudden CPU spike, all triggered within a short window of time.

9.1 Simulation Steps

8. An SSH connection was attempted against the host using a non-existent account and an incorrect password, deliberately producing a failed authentication event.
9. A dummy file was created inside a watched temporary directory to simulate the dropping of a malicious payload.
10. An outbound HTTP request was issued from the same shell session to simulate command-and-control-style network activity.
11. A computationally expensive calculation was executed immediately afterwards to produce an artificial CPU spike from an unrecognised process.

9.2 Observed Results

- The Authentication Logs panel displayed a FAILED entry for the simulated account within seconds of the login attempt.
- The File Integrity Monitor recorded the new file as CREATED under the watched directory.
- The Network Socket States panel showed a new ESTABLISHED connection corresponding to the outbound request.
- The Live Resource Telemetry chart showed an immediate, sharp CPU spike attributed to the unrecognised process.

- The AI Threat Engine correlated all four signals within the same time window and raised a CRITICAL-severity alert describing a correlated, multi-vector attack, rather than four unrelated low-priority notices.

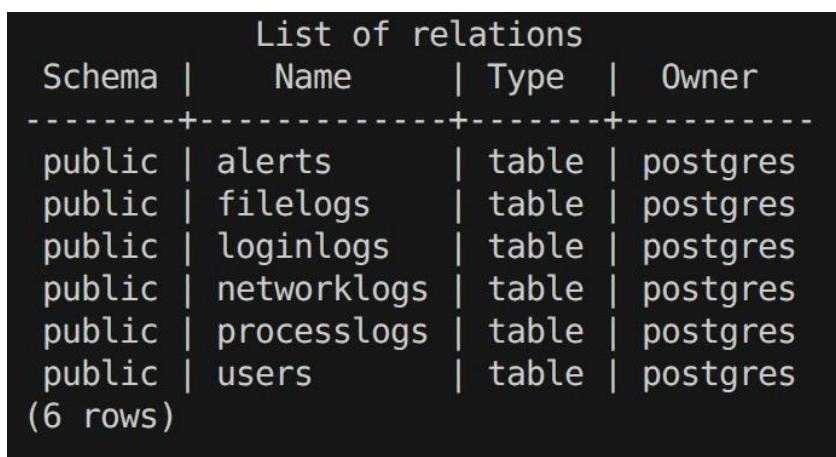
This exercise confirmed that the collectors, the database layer, the correlation logic, and the dashboard operate together correctly as a single pipeline, and that the system meets its original objective of surfacing coordinated threats rather than isolated noise.

10. Results and Screenshots

This section presents visual evidence captured from the completed system, covering the ShaktiDB schema underlying the platform and the dashboard in live operation.

10.1 Database Schema

Figure 2 shows the list of relations present inside the running ShaktiDB instance. All six tables described in Section 5 - Alerts, FileLogs, LoginLogs, NetworkLogs, ProcessLogs, and Users - are present and owned by the postgres role, confirming that the schema was created and deployed exactly as designed.



List of relations			
Schema	Name	Type	Owner
public	alerts	table	postgres
public	filelogs	table	postgres
public	loginlogs	table	postgres
public	networklogs	table	postgres
public	processlogs	table	postgres
public	users	table	postgres
(6 rows)			

Figure 2: List of relations (tables) created in the ShaktiDB instance

10.2 Dashboard in Operation

Figure 3 shows the ShaktiSOC dashboard actively monitoring the host. Several CRITICAL alerts are visible in the AI Threat Engine Alerts panel, each describing an unidentified process whose CPU spike coincided with file modifications and new network sockets, an example of the cross-vector correlation described in Section 8.2. The Live Resource Telemetry chart plots per-process CPU usage over time, while the Authentication Logs, Network Socket States, and File Integrity Monitor panels display the corresponding raw events feeding into those alerts.

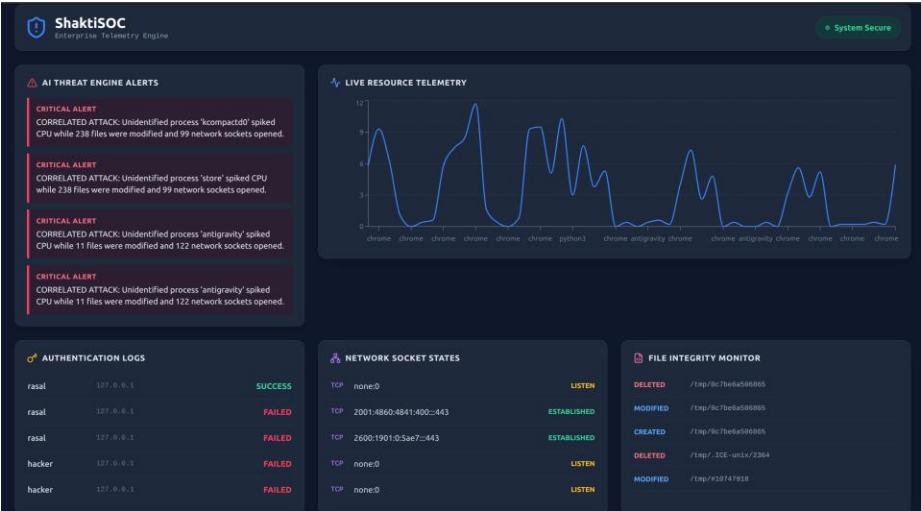


Figure 3: ShaktiSOC dashboard - live view of the AI Threat Engine, telemetry chart, and event logs

Together, Figures 2 and 3 demonstrate a fully operational system: a populated, correctly structured database feeding a live analytics and visualisation layer that is actively detecting and reporting realistic threat scenarios.

11. Development Timeline

The project was carried out over seven weeks, moving from environment setup through to a production-tuned, fully integrated system. Table 5 summarises the focus and outcome of each week, drawn from the weekly progress reports maintained throughout development.

Week	Duration	Focus Area	Outcome
1	19-25 May 2026	Environment setup, GitHub repository, shell scripting assignment	Development environment and workspace ready
2	26 May-1 Jun 2026	ShaktiDB installation, project selection, architecture planning	ShaktiDB installed; ShaktiSOC selected as the project
3	2-8 Jun 2026	Project proposal, detailed architecture, technology evaluation	Design phase completed
4	9-15 Jun 2026	Database schema, process and login collectors	Live telemetry ingestion into ShaktiDB achieved
5	16-22 Jun 2026	Network and file collectors, Flask REST API	Full telemetry collection layer and API completed
6	23-29 Jun 2026	Isolation Forest integration, real-time alerting	AI threat engine operational
7	30 Jun-6 Jul 2026	React dashboard, end-to-end testing	Project completed and validated

Table 5: Seven-week development timeline

12. Challenges and Solutions

A number of practical issues were encountered and resolved over the course of development, in addition to the design decisions already described:

- Python module access for privileged processes - the root user was initially unable to access the pycopg2 library installed in the local user environment; this was resolved by installing the required Python packages system-wide.
- SSH telemetry generation - the host did not initially accept or log SSH connections; installing and enabling openssh-server allowed genuine authentication events, including brute-force attempts, to be captured and logged.
- Duplicate alert generation - without a time boundary, the analytics engine reprocessed historical telemetry on every run; introducing a last_check_time filter restricted analysis to newly generated data only.
- Database transaction errors - failed queries during alert generation occasionally left open transaction blocks; this was resolved by implementing explicit connection rollbacks on failure.
- Alert fatigue - an unrefined anomaly model flagged too many benign spikes; process whitelisting, hard CPU/memory thresholds, and a reduced contamination rate were introduced to ensure only high-fidelity threats were reported.

13. Future Scope

- Replacing or augmenting the Isolation Forest model with deep learning approaches for finer-grained threat classification.
- Adding automated incident response, such as isolating a host or terminating a flagged process without operator intervention.
- Extending the platform to monitor multiple hosts from a single centralised dashboard.
- Integrating external threat intelligence feeds to enrich locally generated alerts with known indicators of compromise.
- Applying AI-assisted forensic analysis to help operators investigate the root cause of an alert more quickly.

Conclusion

ShaktiSOC set out to test whether a database-centred approach could support a practical, real-time security monitoring platform, using ShaktiDB as its foundation. Over seven weeks, the project progressed from initial database installation through independent telemetry collectors, a tuned machine learning detection engine, a documented REST API, and a live React dashboard, culminating in a successful end-to-end validation against a simulated multi-vector attack. Every module identified in the original proposal has been implemented, tested, and confirmed working, and the project is considered successfully delivered. Beyond meeting its own objectives, the finished system also demonstrates that ShaktiDB is capable of handling the continuous, mixed read-write workload typical of a security telemetry pipeline, which was one of the primary questions the project set out to answer.

Author Details

Name: Rasal Musthafa

Programme: B.Tech, Computer Science & Engineering

Project: ShaktiSOC - AI-Powered Security Operations Center using ShaktiDB

Repository: <https://github.com/ras-al/shaktidb-work>